Day 91 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 1 reference

Watch: Building makemore Part 2: MLP (Andrej Karpathy) | Language Model Evaluation and Perplexity (Machine Learning TV)

`day91_causal_language_modeling.py`, segment by segment, in the order `main()` uses things. Day 90's model code is imported and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: h = a context (the characters before) · c = the next character · D = the discount · k = window length

Segment 0 — Setup

Settings, and Day 90's numbers

CODE

```
ORDERS = (1, 2, 3, 4, 5, 6, 7, 8, 9)
DISCOUNTS = (0.5, 0.75, 0.9, 0.95, 0.99)
WINDOWS = (2, 4, 8, 16)
EMB, HID = 24, 256
STEPS, BATCH, LR = 4000, 512, 3e-3
DAY90 = dict(full_context="2.525", addk4="2.554")
```

- `DISCOUNTS = (0.5, 0.75, 0.9, 0.95, 0.99)` — the grid for D; the chosen value must not sit at its edge.
- `DAY90 = dict(full_context="2.525", ...)` — Day 90's results copied from its run output, printed beside today's.

Segment 1 — The chain rule

sequence_logprob: every string's probability

MATH

$$log\ p(x_{1..n}) = \Sigma_t\ log\ p(x_t \mid BOS, x_{1..t-1})$$

CODE

```
def sequence_logprob(P, seqs, causal, h=2):
    BOS = P["E"].shape[0] - 1
    x = np.concatenate([np.full((len(seqs), 1), BOS), seqs[:, :-1]], axis=1)
    n = x.shape[1]
    if causal:
        lg, _ = d90.gpt_forward(x, P, h)
    else:
        z = d90.embed(P["E"], x) + d90.embed(P["pos"], np.arange(n))
        for k in sorted(P["blocks"]):
            z, _ = d90.w12.encoder_block(z, P["blocks"][k], h, np.ones(x.shape, bool))
        lg = d90.output_layer(z, P["Wout"], P["bout"])
    logp = d90.log_softmax(lg[..., :BOS])
    return np.take_along_axis(logp, seqs[..., None], axis=-1)[..., 0].sum(axis=1)
```

- `np.concatenate([np.full((len(seqs), 1), BOS), seqs[:, :-1]], axis=1)` — the inputs: BOS, then the sequence shifted right by one.

- `d90.w12.encoder_block(z, ..., np.ones(x.shape, bool))` — Day 78's block with nothing masked: the non-causal comparison.
- `lg[..., :BOS]` — drops the BOS column, so the softmax is over real tokens only.
- `np.take_along_axis(logp, seqs[..., None], axis=-1)` — picks, at each position, the log-probability of the token that is really there.

demo_chain_rule: listing every string

CODE

```
seqs = np.array(list(itertools.product(range(3), repeat=4)))
for causal in (True, False):
    p = np.exp(sequence_logprob(P, seqs, causal))
    print(f"\n{'causal (Day 90)' if causal else 'no causal mask (Day 78 block)':<30}: sum over 81
        strings = {p.sum():.6f}; "
```

- `itertools.product(range(3), repeat=4)` — every tuple of four tokens from {0, 1, 2}: 81 of them.
- `np.exp(sequence_logprob(...))` — back from log-probabilities to probabilities, to add them up.

Segment 2 — Counting models

KneserNey: counting

MATH

$N_{1+}(\bullet\ g) = |\{c : count(c\ g) > 0\}|$ *(the continuation count)*

CODE

```
class KneserNey:
    def __init__(self, text, order, D=0.75):
        self.n, self.D, self.V = order, D, sorted(set(text))
        self.cnt = [Counter() for _ in range(order + 1)]
        for k in range(1, order + 1):
            self.cnt[k].update(text[i:i + k] for i in range(len(text) - k + 1))
        self.cont = [Counter() for _ in range(order + 1)]
        for k in range(1, order):
            for g in self.cnt[k + 1]:
                self.cont[k][g[1:]] += 1
        self.tab = [None] + [self._tables(k) for k in range(1, order + 1)]
    def _tables(self, k):
        src = self.cnt[k] if k == self.n else self.cont[k]
        tot, kinds = defaultdict(float), defaultdict(int)
        for g, c in src.items():
            tot[g[:-1]] += c; kinds[g[:-1]] += 1
        return src, tot, kinds
```

- `Counter().update(text[i:i + k] for i in ...)` — counts a generator of slices: every k-gram in one pass.
- `self.cont[k][g[1:]] += 1` — one more distinct left neighbour for the string g without its first character.
- `defaultdict(float)` — a dict that starts every new key at 0.0, so totals can be added without checks.
- `tot[g[:-1]] += c` — g[:-1] is the context of g: everything but its last character.

KneserNey.prob and bits: interpolation by recursion

MATH

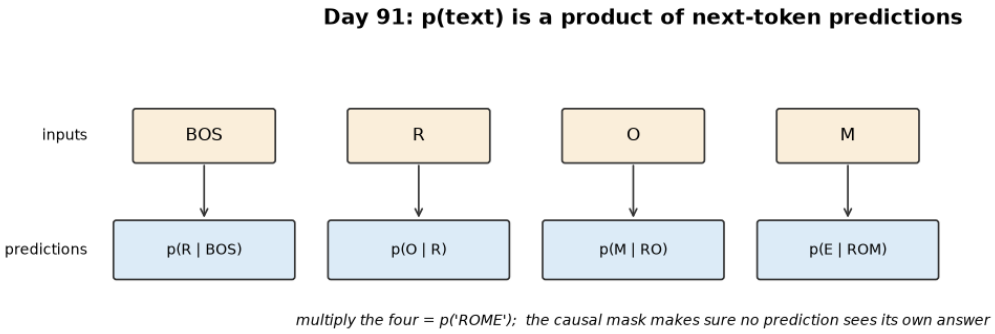
$$p(c \mid h) = \max(\text{count}(h\,c) - D, 0) / \text{count}(h) + (D \cdot N_{1+}(h\,\bullet) / \text{count}(h)) \cdot p(c \mid h')$$

CODE

```
def prob(self, ctx, ch, k=None):
    k = self.n if k is None else k
    if k == 0:
        return 1.0 / len(self.V)
    src, tot, kinds = self.tab[k]
    h = ctx[len(ctx) - (k - 1):] if k > 1 else ""
    if tot[h] == 0:
        return self.prob(ctx, ch, k - 1)
    top = max(src[h + ch] - self.D, 0.0) / tot[h]
    lam = self.D * kinds[h] / tot[h]
    return top + lam * self.prob(ctx, ch, k - 1)
def bits(self, text, start):
    return -np.mean([np.log2(self.prob(text[max(0, i - self.n + 1):i], text[i])) for i in
        range(start, len(text))])
```

- `ctx[len(ctx) - (k - 1):]` — the last k – 1 characters of the context.
- `return self.prob(ctx, ch, k - 1)` — the function calls itself with a shorter context: recursion down to the uniform guess.
- `text[max(0, i - self.n + 1):i]` — the n – 1 characters before position i (fewer at the very start).

SYSTEM DIAGRAM



A text's probability as a product of next-token predictions.

Segment 3 — The window model

WindowLM: the fixed-window neural model

MATH

$$p(c \mid \text{last } k) = \text{softmax}(W_2 \tanh(W_1 [E[x_{t-k}]; \ldots; E[x_{t-1}]] + b_1) + b_2)$$

CODE

```
class WindowLM:
    def __init__(self, V, k, seed=0):
        rng = np.random.RandomState(seed)
        self.k = k
        self.P = dict(E=rng.randn(V, EMB) * 0.1, W1=rng.randn(k * EMB, HID) / np.sqrt(k * EMB),
            b1=np.zeros(HID),
                      W2=rng.randn(HID, V) / np.sqrt(HID) * 0.1, b2=np.zeros(V))
    def logits(self, X):
        self.X = X; self.e = self.P["E"][X].reshape(len(X), -1)
        self.hd = np.tanh(self.e @ self.P["W1"] + self.P["b1"])
        return self.hd @ self.P["W2"] + self.P["b2"]
    def grads(self, X, y):
        lg = self.logits(X); n = len(y)
        loss, dz = d90.softmax_xent(lg, y)
        G = dict(W2=self.hd.T @ dz, b2=dz.sum(axis=0))
        dpre = (dz @ self.P["W2"].T) * (1 - self.hd ** 2)
        G["W1"], G["b1"] = self.e.T @ dpre, dpre.sum(axis=0)
        G["E"] = d90.embed_backward((dpre @ self.P["W1"].T).reshape(n, self.k, EMB), self.P["E"],
            X)
        return loss, G
```

- `self.P["E"][X].reshape(len(X), -1)` — (n, k) ids become (n, k, 24) vectors, then (n, 24k): the k vectors side by side.
- `(1 - self.hd ** 2)` — the derivative of tanh, written with its own output.
- `d90.embed_backward(...)` — Day 85's scatter-add for the shared table.

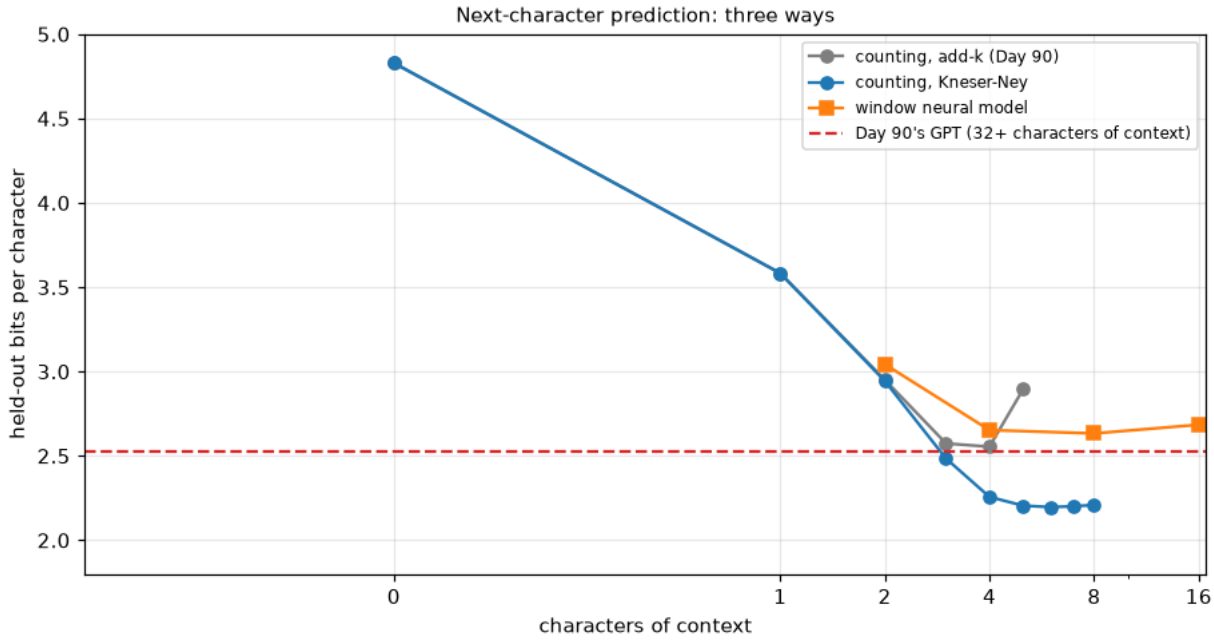
contexts and train_window

CODE

```
def contexts(ids, k, start):
    idx = np.arange(start, len(ids))
    return ids[idx[:, None] - np.arange(k, 0, -1)[None, :]], ids[idx]
def train_window(tr, va, V, k):
    m = WindowLM(V, k); state = {"t": 0, "m": {}, "v": {}}
    rng = np.random.RandomState(1)
    t0 = time.perf_counter()
    for step in range(STEPS):
        i = rng.randint(k, len(tr), BATCH)
        X = tr[i[:, None] - np.arange(k, 0, -1)[None, :]]
        _, G = m.grads(X, tr[i])
        d90.adam_step(m.P, G, state, LR)
    X, y = contexts(va, k, 16)
    lp = d90.log_softmax(np.concatenate([m.logits(X[s:s + 8192]) for s in range(0, len(X),
        8192)]))
    bits = -lp[np.arange(len(y)), y].mean() / np.log(2)
    return bits, sum(a.size for a in m.P.values()), time.perf_counter() - t0
```

- `idx[:, None] - np.arange(k, 0, -1)[None, :]` — broadcasting: for every target position, the k positions before it.
- `d90.adam_step(m.P, G, state, LR)` — Day 90's Adam works on any dictionary of arrays.
- `np.concatenate([m.logits(X[s:s + 8192]) for s in ...])` — scores in chunks, so memory stays small.

REAL GRAPH



Real graph from the script: bits per character against context, three ways.

Quick Syntax Glossary — new or reinforced constructs

- `itertools.product(range(n), repeat=r)` — every tuple of length r.
- `Counter.update(iterable)` — count many items at once.
- `defaultdict(float)` — missing keys start at 0.0.
- `np.take_along_axis(a, idx, axis)` — pick one entry per row by index.
- `a[:, None] - b[None, :]` — an outer difference by broadcasting.
- `recursion` — a function that calls itself on a smaller case, with a stopping case (k == 0).

End of Day 91 syntax reference. For the experiments and what they showed, see the Day 91 complete guide.